Architecture Report — Legal GraphRAG Pipeline

1. Executive Summary

This report describes the architecture, implementation strategy, and design trade-offs of the Legal GraphRAG Pipeline built to process Omani legislation from https://qanoon.om/. The system combines web scraping, graph database modeling, LLM-driven enrichment, vector search, and hybrid retrieval to deliver a production-oriented RAG experience over legal corpora.

2. Problem Statement

Traditional flat-file vector search over legal documents loses structural context, hierarchical relations, and explicit cross-references. Laws are inherently graph-structured: they amend, repeal, and relate to one another, and they cover overlapping legal themes. A GraphRAG approach captures these relationships natively, enabling more contextually aware retrieval and synthesis.

3. System Architecture

3.1 High-Level Data Flow

1. **Acquisition**: A Playwright-based crawler fetches Arabic and English versions of each legal document, evading anti-bot measures through header rotation, randomized delays, and retry logic.
2. **Serialization**: Raw HTML is transformed into clean hierarchical Markdown, preserving legal structure (titles → `#`, sections → `##`, articles → `###`, tables → Markdown tables).
3. **Graph Ingestion**: Documents are stored as central `Document` nodes with language-specific content properties.
4. **Enrichment**: An LLM extracts legal topics; semantic chunking splits documents into retrievable segments.
5. **Vectorization**: Topics and chunks are embedded and indexed for similarity search.
6. **Retrieval**: A hybrid search client combines BM25 keyword matching, dense vector search, graph traversal, and cross-encoder reranking to produce synthesized answers.

3.2 Graph Schema

The graph schema follows the simplified schema principle mandated by the assessment brief:

- **Document nodes** contain all language variants as properties (`contentAr`, `contentEn`).
- **Topic nodes** represent LLM-extracted legal themes.
- **Chunk nodes** represent semantic text segments.
- **Relationships** capture legal cross-references (`AMENDS`, `REPEALS`) and associations (`HAS_TOPIC`, `HAS_CHUNK`).

This design minimizes join complexity and aligns with the requirement that translations not

be split into separate linked nodes.

4. Component Design

4.1 Crawler & State Manager

The crawler is built on Playwright to handle JavaScript-rendered pages. It implements:

- Custom headers and user-agent rotation.
- Randomized request pacing.
- Exponential backoff for transient failures.
- JSON-based checkpointing to enable resumability after interruption.

4.2 Markdown Transformer

The transformer uses `markdownify` and BeautifulSoup to:

- Convert headings into hierarchical Markdown headers.
- Preserve tabular data in Markdown table syntax.
- Remove navigation, advertisements, scripts, and stylistic markup.

4.3 Graph Database Layer

Neo4j Community Edition is deployed via Docker Compose. The ingestion layer uses parameterized Cypher queries for safe and efficient batch inserts. Vector indexes are created on `Topic` and `Chunk` embeddings using Neo4j's native vector search capabilities.

4.4 LLM Agents

The topic extraction agent sends consolidated markdown content to a local LLM via Ollama (`qwen2.5:14b`) with a structured prompt requesting a JSON array of topics. The chunking agent uses `RecursiveCharacterTextSplitter` with configurable chunk size and overlap. Using a local model eliminates API costs and ensures the pipeline operates fully offline.

4.5 Vector Operations

Embeddings are generated locally using Ollama's `qwen3-embedding:4b` model, which is optimized for multilingual retrieval including Arabic and English legal text. A topic merger script computes cosine similarity between topic embeddings and consolidates synonymous topics above a configurable threshold (default 0.88).

4.6 Relationship Extractor

Legal cross-references (`AMENDS` and `REPEALS`) are extracted from document content using language-specific regex patterns. Arabic-Indic numerals are normalized to Western numerals before matching. Extracted references are resolved against the graph using the target document `number` property and persisted as typed Neo4j relationships. In the current 100-document deployment, two `AMENDS` relationships were created; `REPEALS` references were detected but their targets are not present in the sample.

4.7 Search Client

The search client implements a multi-stage retrieval pipeline:

1. **Candidate Generation**: Weighted combination of BM25 sparse scores and dense vector similarity scores.
2. **Topological Context Expansion**: Graph traversal retrieves parent Document metadata, related Topics, and any `AMENDS/REPEALS` cross-references for each candidate chunk.
3. **Cross-Encoder Reranking**: A local reranker model rescores expanded candidates against the query.
4. **LLM Synthesis with Fallback**: The primary synthesis model (`qwen2.5:14b`) is tried first; if it fails (e.g., GPU memory exhaustion), the client automatically retries with a configured fallback model (`qwen2.5:7b`) so queries remain answerable.

5. Performance & Scaling Considerations

- **Embedding Generation**: Batch processing and local models reduce cost and latency.
- **Graph Traversal**: Neo4j's native graph engine supports efficient multi-hop queries for context expansion.
- **Search Latency**: Cross-encoder reranking on CPU is the main latency bottleneck for interactive queries; for a demo setup it is acceptable, but it can be disabled or replaced with a smaller model for faster responses.
- **Scalability**: For 1,000,000+ articles, the ingestion layer would be distributed via a task queue (e.g., Celery/RQ), embeddings would be computed on GPU workers, and Neo4j would be clustered or sharded by legal domain.

6. Deployment Results

The pipeline has been executed end-to-end on real data from qanoon.om. The resulting knowledge graph contains:

Metric	Value
Real qanoon.om documents	100
Semantic chunks	2,202
Extracted topics	173
AMENDS relationships	2
Embedding dimensions	2560 (qwen3-embedding:4b)
LLM	qwen2.5:14b via Ollama (qwen2.5:7b fallback)

Sample/synthetic data was removed so the graph contains only real Omani legislation. Hybrid search successfully answers Arabic legal questions and cites the relevant Royal Decrees and articles.

7. Trade-offs & Rationale

Decision	Alternative	Rationale
Neo4j for graph + vector	Separate vector DB (Pinecone, Weaviate)	Simplifies deployment and traversal between vector results and graph context.
Ollama for LLM (`qwen2.5:14b`)	OpenAI API	Fully free, offline, no API credits required.

qwen3-embedding:4b via Ollama	OpenAI/text-embedding-3-small	Local embedding generation optimized for Arabic and English legal text.
Automatic LLM fallback	Single model	Keeps demo/query interface reliable when the primary model exhausts limited GPU memory.
Regex cross-reference extraction	LLM-based extraction	Fast, deterministic, and avoids additional LLM calls during ingestion.
One Document node	Separate nodes per language	Directly follows exam schema requirement and reduces query complexity.
Resumable checkpointing	Full in-memory crawl	Essential for achieving 100% coverage over unreliable network conditions.

8. Conclusion

The Legal GraphRAG Pipeline demonstrates a clean, modular, and scalable approach to building a graph-enhanced RAG system over legal corpora. It prioritizes engineering robustness, schema compliance, and evaluation alignment while providing clear extension points for future optimization. The successful end-to-end deployment on 100 real qanoon.om documents validates the architecture and confirms that the system can retrieve and synthesize accurate legal answers in Arabic.